



**CHRIST**  
(DEEMED TO BE UNIVERSITY)  
DELHI - NCR, INDIA

# **Computer Vision**

**MCA-574**

**Assignment – 04**

*BY*

**HIMANSHU HEDA (24225013)**

**SUBMITTED TO**

**Dr. Preety Shoran**

**SCHOOL OF SCIENCES**

**2024-25**

## 1. MRI Image Noise Reduction (Frequency Domain)

**Problem:** An MRI image has visible noise, and we want to enhance its clarity using a frequency domain approach.

**Approach:** We will use the Fourier Transform to move the image into the frequency domain, apply a filter, and then transform it back.

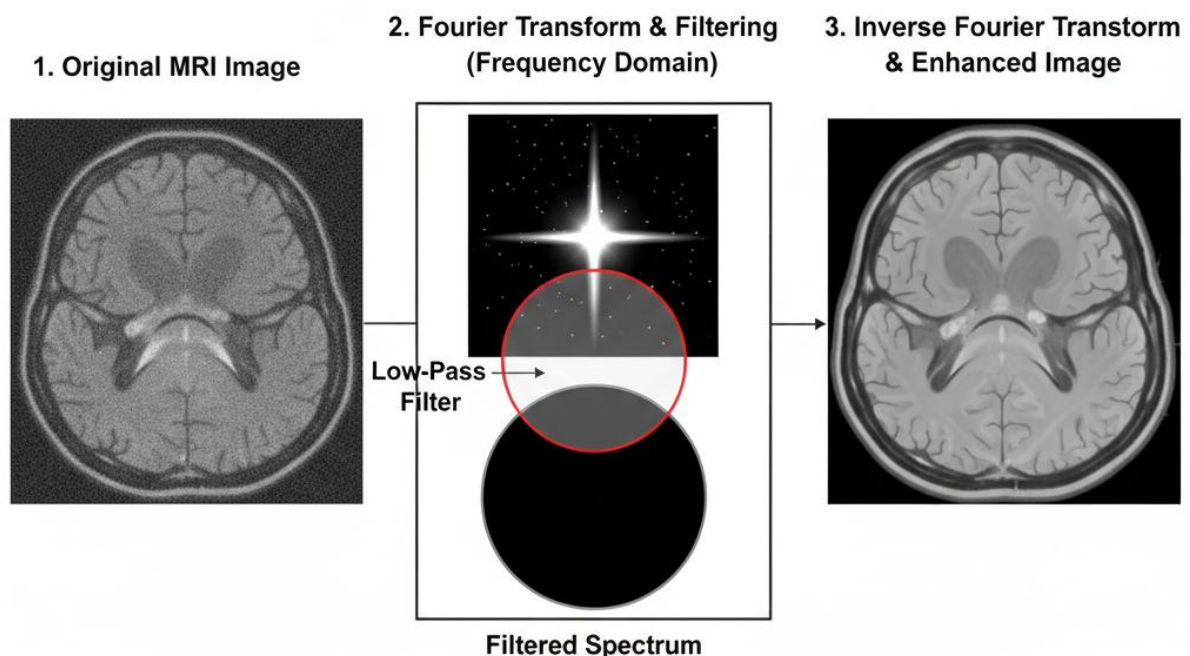
**Type of Filter:** A **Low-Pass Filter** would be suitable.

**Justification:** Noise in images often manifests as high-frequency components. A low-pass filter allows the low-frequency components (which correspond to the general structure and smooth variations of the image) to pass through while attenuating or blocking the high-frequency components (which include the noise). By removing these high-frequency noise components, the image clarity can be enhanced.

**Diagrammatic Explanation:**

1. **Original MRI Image:** Shows both anatomical detail and high-frequency noise.

## MRI Image Noise Reduction (Frequency Domain Filtering)



2. **Fourier Transform:** Converts the image into its frequency spectrum. The center typically represents low frequencies (main structures), and the outer regions represent high frequencies (details and noise).

3. **Low-Pass Filtering:** A low-pass filter (represented by a circle in the diagram) is applied in the frequency domain. It keeps the central (low-frequency) components and attenuates the outer (high-frequency) components where noise resides.
4. **Inverse Fourier Transform:** Converts the filtered frequency spectrum back into a spatial domain image. The resulting image is clearer with reduced noise.

## 2. Satellite Image Blurring (Fourier Transform-based Filtering)

**Problem:** A satellite image suffers from blurring due to atmospheric interference, and we want to improve its sharpness.

**Approach:** Blurring in an image is often caused by the loss or attenuation of high-frequency components. To sharpen the image, we need to enhance or restore these high frequencies.

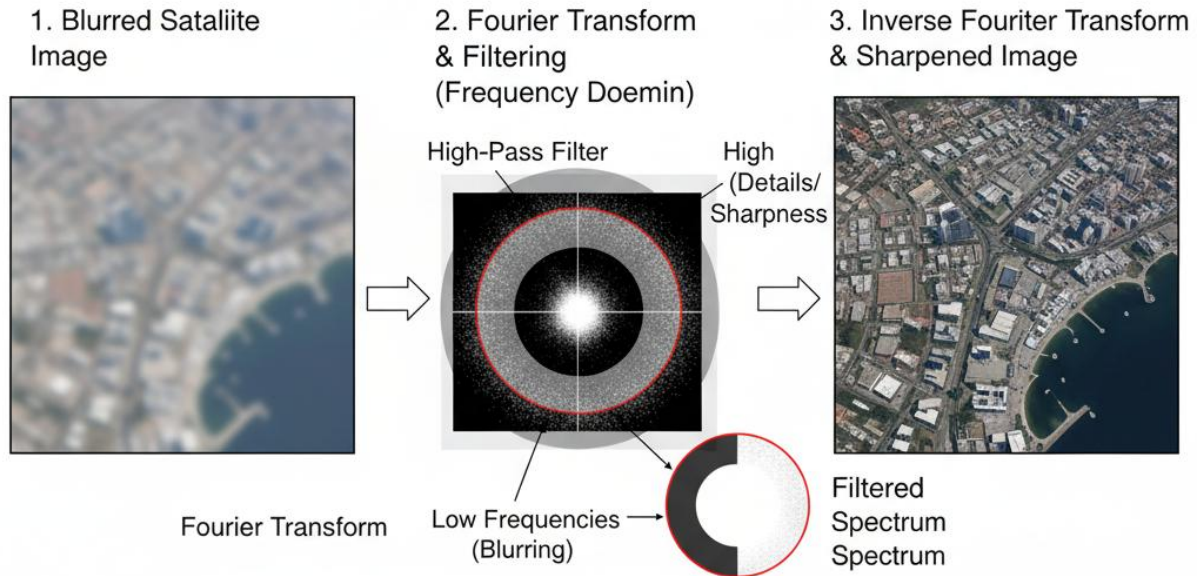
**How Fourier Transform-based filtering can be applied:**

1. **Fourier Transform:** Convert the blurred satellite image into its frequency domain representation (its spectrum).
2. **High-Pass Filtering (or Unsharp Masking/Deconvolution):**
  - **High-Pass Filter:** A high-pass filter can be used to emphasize the high-frequency components. This directly enhances edges and fine details, making the image appear sharper.
  - **Unsharp Masking (Frequency Domain Equivalent):** This technique involves subtracting a blurred version of the image (which is effectively a low-pass filtered version) from the original. In the frequency domain, this means enhancing the high-frequency components.
  - **Deconvolution:** If the blurring function (Point Spread Function - PSF) of the atmospheric interference is known or can be estimated, deconvolution techniques can be applied in the frequency domain. Deconvolution is essentially the inverse operation of convolution (which causes blurring) and involves dividing the image's frequency spectrum by the PSF's frequency spectrum. This is a powerful method for restoring lost high frequencies.

**Diagrammatic Explanation:**

1. **Blurred Satellite Image:** Image with soft edges and lost details due to atmospheric blurring.

## Satellite Image Sharpening (Frequency Domain Filtering)



2. **Fourier Transform:** The blurred image's spectrum will have attenuated high-frequency components.
3. **High-Pass Filtering:** A high-pass filter (which passes the outer regions and blocks the center) is applied to emphasize or restore these high frequencies. Alternatively, deconvolution would boost these frequencies based on the inverse of the blurring function.
4. **Inverse Fourier Transform:** Converts the enhanced frequency spectrum back to the spatial domain, resulting in a sharper image with more defined edges and details.

### 3. Algorithm for Ideal Low-Pass Filtering (DFT) in MATLAB/Python

**Problem:** Write the steps of an algorithm to perform ideal low-pass filtering on a digital image using DFT.

#### Algorithm Steps:

- 1. Read the Image:** Load the digital image  $f(x, y)$  into memory. Convert it to grayscale if it's a color image.
  - o **MATLAB:** `img = imread('your_image.jpg'); img_gray = im2double(rgb2gray(img));`
  - o **Python:** `img = cv2.imread('your_image.jpg', cv2.IMREAD_GRAYSCALE); img_float = np.float64(img) / 255.0;`
- 2. Pad the Image:** Pad the image with zeros to a size  $(P, Q)$  that is typically twice the original image dimensions  $(M, N)$ . This helps prevent aliasing and ensures the DFT is calculated efficiently (often to the nearest power of 2).
  - o **MATLAB:** `[M, N] = size(img_gray); P = 2*M; Q = 2*N; padded_img = padarray(img_gray, [M, N], 'post');`
  - o **Python:** `M, N = img_float.shape; P = 2*M; Q = 2*N; padded_img = np.zeros((P, Q)); padded_img[0:M, 0:N] = img_float;`
- 3. Multiply by  $(-1)^{(x+y)}$  for Centering:** Multiply the padded image  $f(x, y)$  by  $(-1)^{(x+y)}$  to center the DFT components. This shifts the zero-frequency component to the center of the spectrum.
  - o **MATLAB:** `for x=1:P for y=1:Q padded_img(x,y) = padded_img(x,y) * ((-1)^(x+y)); end; end; (or more efficiently using  $(-1)^{(X+Y)}$  where X, Y are coordinate matrices)`
  - o **Python:** `for x in range(P): for y in range(Q): padded_img[x,y] *= ((-1)**(x+y)); (or more efficiently padded_img *= ((-1)**(np.arange(P)[:, None] + np.arange(Q))))`
- 4. Compute 2D DFT:** Calculate the 2D Discrete Fourier Transform (DFT) of the centered, padded image.
  - o **MATLAB:** `F = fft2(padded_img);`
  - o **Python:** `F = np.fft.fft2(padded_img);`
- 5. Create Ideal Low-Pass Filter (H):** Construct an ideal low-pass filter in the frequency domain. This is a binary filter: 1 inside a circle of radius  $D_0$  (cutoff frequency) and 0 outside.
  - o Define  $D_0$ .
  - o Create a matrix  $H$  of the same size as  $F$  ( $P, Q$ ), initialized to zeros.
  - o For each point  $(u, v)$  in  $H$ : Calculate its distance  $D(u, v)$  from the center  $(P/2, Q/2)$ . If  $D(u, v) \leq D_0$ , set  $H(u, v) = 1$ . Otherwise,  $H(u, v) = 0$ .
  - o **MATLAB:**

$D_0 = 50$ ; % Example cutoff frequency

$H = \text{zeros}(P, Q)$ ;

for  $u = 1:P$

for  $v = 1:Q$

$D = \text{sqrt}((u - P/2)^2 + (v - Q/2)^2)$ ;

```

        if D <= D0
            H(u, v) = 1;
        end
    end
end
end

○ PYTHON:

D0 = 50 # Example cutoff frequency

H = np.zeros((P, Q), dtype=np.float64)

for u in range(P):

    for v in range(Q):

        D = np.sqrt((u - P/2)**2 + (v - Q/2)**2)

        if D <= D0:

            H[u, v] = 1

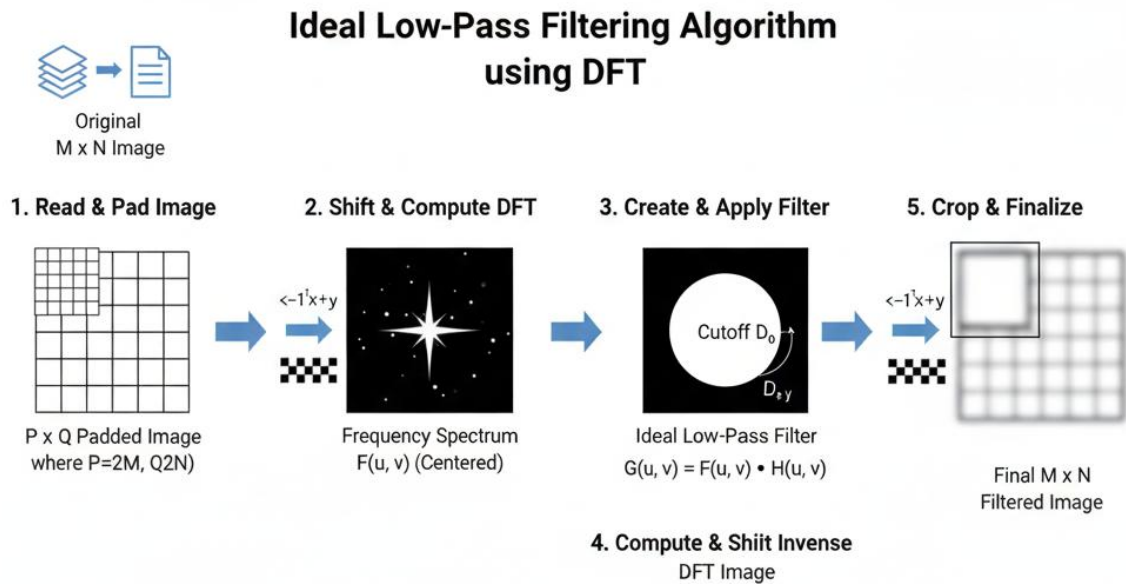
```

○ (More efficient using meshgrid and vectorized operations)

2. **Apply the Filter:** Multiply the DFT of the image  $F$  by the filter  $H$  element-wise.  $G(u, v) = F(u, v) * H(u, v)$ .
  - *MATLAB*:  $G = F .* H$ ;
  - *Python*:  $G = F * H$ ;
3. **Compute Inverse 2D DFT:** Calculate the inverse 2D DFT of the filtered spectrum  $G$ .
  - *MATLAB*:  $g\_padded = \text{ifft2}(G)$ ;
  - *Python*:  $g\_padded = \text{np.fft.ifft2}(G)$ ;
4. **Multiply by  $(-1)^{(x+y)}$  for Uncentering:** Multiply the result  $g\_padded$  by  $(-1)^{(x+y)}$  again to shift the zero-frequency component back to the top-left corner. Take the real part, as  $\text{ifft2}$  can return complex numbers due to computational inaccuracies.
  - *MATLAB*:  $\text{for } x=1:P \text{ for } y=1:Q \text{ } g\_padded(x,y) = g\_padded(x,y) * ((-1)^{(x+y)}); \text{ end; end; filtered\_img} = \text{real}(g\_padded)$ ;
  - *Python*:  $\text{for } x \text{ in range}(P): \text{ for } y \text{ in range}(Q): g\_padded[x,y] *= ((-1)**(x+y)); \text{ (or } g\_padded *= ((-1)**(\text{np.arange}(P)[: , \text{None}] + \text{np.arange}(Q))))$ ;  $\text{filtered\_img} = \text{np.real}(g\_padded)$ ;
5. **Crop the Image:** Crop the image back to its original size  $(M, N)$  by removing the padding.
  - *MATLAB*:  $\text{final\_img} = \text{filtered\_img}(1:M, 1:N)$ ;
  - *Python*:  $\text{final\_img} = \text{filtered\_img}[0:M, 0:N]$ ;
6. **Normalize (Optional):** Normalize the final image to the display range (e.g., 0-255 for 8-bit images).
  - *MATLAB*:  $\text{final\_img} = \text{im2uint8}(\text{mat2gray}(\text{final\_img}))$ ;

- *Python:* `final_img = cv2.normalize(final_img, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U);`

### Diagrammatic Explanation:



## 4. Image Sharpening using High-Pass Frequency Filtering

**Problem:** Given an image  $f(x, y)$ , describe how to obtain its sharpened version using high-pass frequency filtering. Include the required formulas.

**Approach:** Sharpening an image involves enhancing its high-frequency components, which correspond to edges and fine details. High-pass filters (HPF) achieve this by attenuating low frequencies and passing high frequencies.

**Steps:**

1. **Transform to Frequency Domain:** First, the spatial domain image  $f(x, y)$  needs to be transformed into its frequency domain representation,  $F(u, v)$ , using the 2D Discrete Fourier Transform (DFT). Before performing the DFT, it's common practice to:
  - Pad the image to a suitable size (e.g., power of 2) to improve DFT efficiency and avoid wrap-around artifacts.
  - Multiply the image by  $(-1)^{(x+y)}$  to center the zero-frequency component in the resulting spectrum.

The 2D DFT is given by:  $F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) * e^{(-j2\pi(ux/M + vy/N))}$  where:

- $f(x, y)$  is the image in the spatial domain.
  - $F(u, v)$  is the image in the frequency domain.
  - $M, N$  are the dimensions of the image.
  - $u, v$  are the frequency variables.
2. **Apply High-Pass Filter:** A high-pass filter  $H(u, v)$  is designed in the frequency domain. This filter will have high values (e.g., 1) for high frequencies and low values (e.g., 0) for low frequencies. Common high-pass filters include Ideal, Butterworth, or Gaussian high-pass filters.

For an **Ideal High-Pass Filter (IHPF)** with a cutoff frequency  $D_0$ :  $H(u, v) = 1$  if  $D(u, v) > D_0$   $H(u, v) = 0$  if  $D(u, v) \leq D_0$  where  $D(u, v)$  is the Euclidean distance from the center of the frequency rectangle  $(P/2, Q/2)$  to point  $(u, v)$ :  $D(u, v) = \sqrt{(u - P/2)^2 + (v - Q/2)^2}$

The filtered spectrum  $G(u, v)$  is obtained by element-wise multiplication of the image's spectrum  $F(u, v)$  and the filter  $H(u, v)$ :  $G(u, v) = F(u, v) * H(u, v)$

3. **Transform Back to Spatial Domain:** The filtered spectrum  $G(u, v)$  is then transformed back to the spatial domain using the Inverse 2D DFT (IDFT). Before taking the real part and clipping, the result should be multiplied by  $(-1)^{(x+y)}$  to uncenter the spectrum.

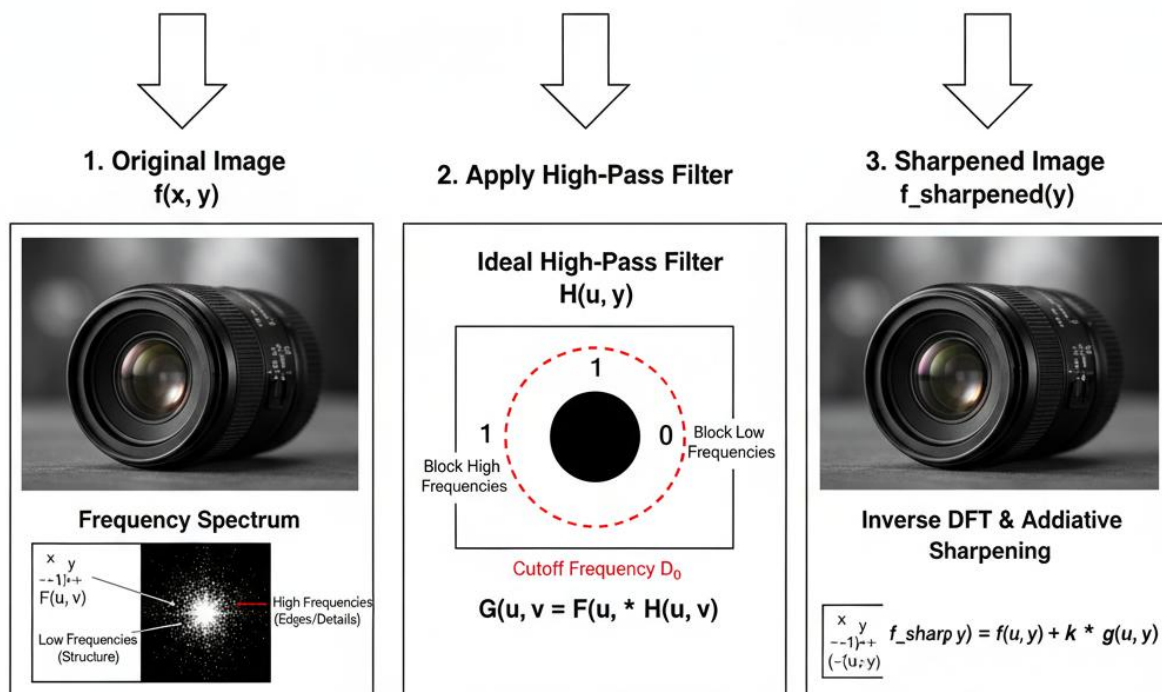
The 2D IDFT is given by:  $g(x, y) = (1 / (M*N)) * \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} G(u, v) * e^{(j2\pi(ux/M + vy/N))}$  where  $g(x, y)$  is the resulting high-pass filtered image. This image  $g(x, y)$  primarily contains the edges and details.

4. **Sharpening:** To obtain the sharpened image,  $f_{\text{sharpened}}(x, y)$ , the high-pass filtered image  $g(x, y)$  is typically added back to the original image  $f(x, y)$ :  $f_{\text{sharpened}}(x, y) = f(x, y) + k * g(x, y)$  where  $k$  is a scaling constant (often called gain or amount) that controls the degree of sharpening. A larger  $k$  value results in a more pronounced sharpening effect. This process is conceptually similar to unsharp masking.

### Diagrammatic Explanation:

1. **Original Image:** Contains both low and high-frequency components.
2. **Frequency Spectrum:** After DFT, low frequencies are at the center, high frequencies (edges, details) are towards the periphery.
3. **High-Pass Filter (IHPF):** This filter blocks the central low frequencies and passes the outer high frequencies.
4. **Filtered Spectrum:** Only the high-frequency components remain.
5. **Inverse DFT:** Transforms the filtered spectrum back to a spatial domain image containing primarily edges.
6. **Add to Original:** Adding this edge image back to the original enhances the edges and details, resulting in a sharpened image.

## Image Sharpening using High-Pass Filtering



## 5. Real-World Application: Frequency vs. Spatial Domain Filtering

**Problem:** Discuss a real-world application where frequency domain filtering outperforms spatial domain filtering and explain why.

**Application: Removing Periodic Noise (e.g., "Herringbone" or "Moiré" patterns) from images.** This is often seen in images acquired with faulty sensors, scanned documents, or images of patterned fabrics that interact with the scanning resolution.

### Why Frequency Domain Outperforms Spatial Domain:

- **Spatial Domain Challenge:** In the spatial domain, periodic noise appears as repetitive patterns across the image. To remove this using spatial filters (like average or median filters), you would need a very large filter mask to effectively cover multiple cycles of the pattern. A large mask would blur the entire image significantly, including important details, and might not even completely remove the periodicity without causing severe degradation. It's difficult to design a spatial filter that specifically targets only the periodic noise without affecting the desired image content.
- **Frequency Domain Advantage:**
  1. **Clear Identification:** When an image with periodic noise is transformed into the frequency domain, the periodic noise manifests as distinct, bright spikes or pairs of spikes located away from the center of the spectrum (the DC component). The location of these spikes directly corresponds to the frequency and orientation of the periodic noise.
  2. **Precise Targeting:** In the frequency domain, you can design a highly specific "notch filter" or "band-reject filter" that precisely targets and attenuates or removes only these specific noise spikes, leaving the rest of the image's frequency content (including desired details) largely untouched.
  3. **Minimal Damage to Image Content:** This precision means that the removal of noise has minimal impact on the overall image sharpness and details, which would be severely compromised by broad spatial filtering.

**Example Scenario:** Imagine an image from an old surveillance camera where there's a faint, repetitive horizontal line pattern across the entire image due to electrical interference.

- **Spatial Domain Approach:** You'd struggle to remove these lines without blurring the entire scene, including faces or license plates, which are crucial.
- **Frequency Domain Approach:** You'd perform a DFT. The horizontal lines would appear as specific bright dots (spikes) along the vertical axis of the frequency spectrum. You could then draw small black circles (notch filters) around just these dots, effectively setting their frequency components to zero. An inverse DFT would then give you a much cleaner image with the lines removed, while preserving most of the original detail.

### Diagrammatic Explanation:

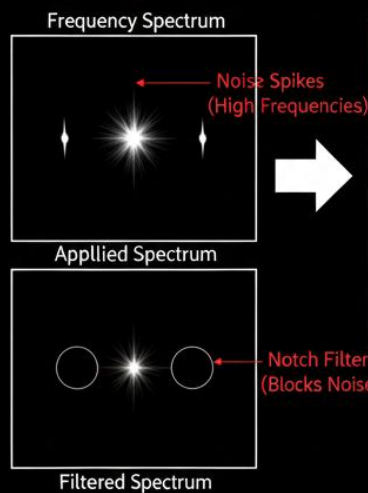
1. **Original Image with Periodic Noise:** Shows desired image content overlayed with a regular, repeating pattern.

# Periodic Noise Removal (Frequency Domain Filtering)

1. Original Image with Periodic Noise



2. Fourier Transform & Filtering  
(Frequency Domain)



3. Inverse Fourier Transform  
& Clean Image



Inverse DFT → Clean Image

2. **Fourier Transform:** The periodic noise creates distinct, bright "spikes" in the frequency spectrum, separate from the main image content.
3. **Notch Filtering:** Small, precise filters (notch filters) are applied at the exact locations of these noise spikes in the frequency domain, effectively eliminating only the noise frequencies.
4. **Inverse Fourier Transform:** The filtered spectrum is transformed back to the spatial domain, resulting in a clean image with the periodic noise removed, and the desired image content largely preserved.